

Sistemi Distribuiti e Cloud Computing

Project B6: Sistema di Rate Limiting Distribuito in Go

Francesco Bernardini

Matricola: 0338264

Laurea Magistrale in Ingegneria Informatica

Università degli studi di Roma Tor Vergata

francesco.bernardini.01@students.uniroma2.eu

Abstract— Questo documento descrive la progettazione e l'implementazione di un sistema di rate limiting distribuito, pensato per proteggere API da attacchi di tipo DoS e per controllare le quote d'uso dei singoli client, senza introdurre un punto di controllo centralizzato. Il sistema è composto da tre tipologie di nodo indipendenti: un livello edge stateless, un livello di aggregazione stateful e un servizio di analisi asincrono.

I. INTRODUZIONE

Per rispondere all'esigenza di limitare il traffico in modo scalabile, senza introdurre un singolo punto di controllo che rischierebbe di diventare un collo di bottiglia o un singolo punto di guasto, è stata scelta un'architettura distribuita, in cui il conteggio delle richieste è gestito da più nodi che cooperano tra loro invece che da un'unica componente centralizzata.

Il sistema implementato è composto da tre tipologie di nodi. Il nodo edge riceve la richiesta da parte dell'utente e ha il compito di inoltrarla a uno degli aggregator. Quest'ultimo tipo di nodo riceve la richiesta dell'utente da parte dell'edge e la accetta qualora l'utente non abbia terminato il proprio budget di token, altrimenti la rifiuta. In modo asincrono lavora il terzo tipo di nodo, l'analytics, che riceve le notifiche delle richieste rifiutate e segnala quando un client supera una soglia critica di violazioni in un dato intervallo di tempo.

Il resto del documento è organizzato come segue: la Sezione II descrive l'architettura del sistema e le scelte progettuali adottate; la Sezione III illustra l'implementazione realizzata; la Sezione IV presenta la piattaforma software utilizzata e la metodologia di testing; la Sezione V riporta i risultati dei test effettuati; la Sezione VI discute infine le limitazioni riscontrate.

II. ARCHITETTURA DEL SISTEMA

A. Panoramica

Il sistema è composto da tre tipologie di nodo indipendenti, che comunicano tra loro secondo due modalità diverse a seconda della natura dell'interazione. La comunicazione tra edge e aggregator, e tra aggregator tra loro, avviene tramite RPC sincrona (gRPC/Protobuf), perché in entrambi i casi il nodo chiamante ha bisogno di una risposta immediata prima di poter proseguire. La comunicazione tra edge e analytics avviene invece in modo asincrono tramite un broker di messaggistica (NATS), perché analytics non partecipa in alcun modo alla decisione di accettare o rifiutare una richiesta, e può quindi essere completamente disaccoppiato dal percorso critico che serve il client. Questa

distinzione, sincrona dove serve una decisione immediata e asincrona dove basta notificare un evento, è la motivazione alla base della scelta di due pattern di comunicazione distinti invece di uno solo per l'intero sistema.

B. Edge

Il nodo edge è stateless e rappresenta il punto di ingresso del sistema: riceve le richieste HTTP dei client, ne verifica la quota interrogando l'aggregator competente, ed inoltra in modo asincrono le richieste rifiutate ad analytics.

La prima scelta progettuale rilevante è l'instradamento deterministico dei client verso gli aggregator, ottenuto tramite una funzione di hash (FNV-1a) applicata al client id. Questa scelta è necessaria per correttezza: se le richieste di uno stesso client potessero raggiungere aggregator diversi in modo casuale (ad esempio con un semplice round-robin), ciascun nodo manterrebbe un conteggio parziale e indipendente per lo stesso client, rendendo il limite di richieste facilmente aggirabile e vanificando lo scopo stesso del rate limiting. L'hashing garantisce invece che ogni client venga sempre indirizzato allo stesso nodo primario, salvo il caso di guasto.

La seconda scelta è l'introduzione di un circuit breaker tra edge e ciascun aggregator. Gli aggregator, essendo processi distinti su nodi potenzialmente diversi, possono guastarsi o diventare temporaneamente irraggiungibili. Senza un meccanismo di protezione, ogni richiesta diretta a un nodo guasto pagherebbe per intero il tempo di attesa del timeout di rete, per ogni singola richiesta, appesantendo inutilmente la latenza percepita dal client. Il circuit breaker rileva i fallimenti ripetuti verso un nodo, smette temporaneamente di interrogarlo, e instrada le richieste verso il successivo nodo disponibile nell'anello di hashing. Solo nel caso estremo in cui tutti gli aggregator risultino irraggiungibili, il sistema privilegia la disponibilità sulla precisione, accettando la richiesta in modalità degradata (fail-open) piuttosto che interrompere il servizio.

C. Aggregator

Il nodo aggregator è stateful e mantiene in memoria lo stato dei token residui per ciascun client, secondo il modello del token bucket: ogni client dispone di una capacità massima di token, che si ricarica in modo continuo nel tempo; una richiesta viene accettata se è disponibile almeno un token, che viene contestualmente consumato, altrimenti viene rifiutata.

Lo spazio dei client è partizionato orizzontalmente tra i nodi aggregator (sharding) tramite la stessa funzione di hash

usata dall'edge. Questa scelta permette al carico di memoria e di calcolo di crescere in modo proporzionale al numero di nodi aggiunti al sistema, invece di gravare interamente su un unico nodo centrale.

Poiché lo stato di ciascun client risiede, in condizioni normali, su un solo nodo, un guasto del nodo primario comporterebbe la perdita del relativo conteggio. Per questo motivo gli aggregator si scambiano periodicamente, tramite un protocollo di gossip, lo stato dei bucket modificati di recente. In questo modo, quando il circuit breaker dell'edge instrada una richiesta verso un nodo secondario, quest'ultimo dispone già di una copia recente dello stato del client, invece di trattarlo come un client mai visto prima con budget pieno, il che comprometterebbe la precisione del conteggio proprio nel momento del guasto. Rispetto a una soluzione alternativa, come l'uso di un database condiviso esterno, il gossip evita di introdurre un nuovo componente centralizzato che reintrodurrebbe il collo di bottiglia che l'architettura a più nodi mira ad eliminare, al costo di una consistenza solo eventuale, e non immediata, tra le repliche.

D. Analytics

Il nodo analytics è stateless e riceve, tramite sottoscrizione al broker NATS, gli eventi corrispondenti alle richieste rifiutate dall'edge. Per ciascun client, analytics conta il numero di violazioni ricevute in una finestra temporale, ed emette un avviso quando tale numero supera una soglia configurata. L'uso di una soglia calcolata su una finestra, invece di un avviso alla prima violazione, permette di distinguere un client occasionalmente sopra le righe da un client che sta sistematicamente tentando di superare il limite imposto.

La scelta di ricevere gli eventi in modo asincrono, anziché tramite una chiamata sincrona da parte dell'edge, deriva dal fatto che analytics non influisce in alcun modo sulla decisione di accettare o rifiutare una richiesta: rendere l'edge dipendente, anche solo per un log, dalla disponibilità o dalla velocità di analytics introdurrebbe un accoppiamento e una latenza aggiuntiva sul percorso critico senza alcun beneficio corrispondente.

E. Pattern architetturali adottati

In sintesi, il sistema implementa i seguenti pattern architetturali: RPC sincrona (gRPC/Protobuf) per le decisioni che richiedono una risposta immediata; messaggistica asincrona event-driven (NATS) per il disaccoppiamento tra edge e analytics; circuit breaker per l'isolamento dei guasti verso gli aggregator; sharding con replica tramite gossip per la distribuzione e la tolleranza ai guasti dello stato distribuito. A questi si aggiunge, come estensione oltre i requisiti minimi, il distributed tracing (OpenTelemetry e Jaeger), utilizzato per correlare i tempi di elaborazione di una stessa richiesta lungo l'intera catena dei nodi.

III. IMPLEMENTAZIONE REALIZZATA

Se la sezione precedente ha descritto l'architettura a livello di pattern, questa sezione entra nel dettaglio delle scelte implementative con cui ciascun pattern è stato realizzato concretamente all'interno del progetto.

A. Token bucket e gestione della quota

Il meccanismo di rate limiting vero e proprio è realizzato con un algoritmo a token bucket mantenuto da ogni nodo aggregator per ciascun client. A ogni client è associato un bucket con una capacità massima (Capacity) e un tasso di

ricarica (RefillRate, espresso in token al secondo); nella configurazione finale la capacità è pari a 5 token con un tasso di 0.5 token/secondo, ovvero un token restituito ogni 2 secondi. Ad ogni richiesta l'aggregator calcola il tempo trascorso dall'ultimo accesso al bucket, ricarica i token maturati nel frattempo (fino al tetto massimo) e, se è disponibile almeno un token, lo consuma e concede la richiesta; in caso contrario la rifiuta. Questa logica è incapsulata in una struttura dati unica per bucket che tiene traccia del numero di token correnti e del timestamp dell'ultimo aggiornamento, così da rendere il calcolo indipendente dalla frequenza con cui le richieste arrivano.

B. Sharding tramite hashing consistente

Il nodo edge non mantiene alcuno stato relativo alle quote; il suo compito è instradare deterministicamente ogni client verso lo stesso nodo aggregator nel tempo, in modo che i bucket non vengano mai letti o scritti da più nodi diversi. L'instradamento è realizzato calcolando l'hash FNV-1a dell'identificativo del client e usando il risultato per selezionare, per modulo, uno degli aggregator noti. L'elenco degli aggregator disponibili è configurato tramite un'unica lista di indirizzi (AGGREGATOR_ADDRESSES), condivisa sia dall'edge per lo sharding sia da ciascun aggregator per conoscere i propri peer nel gossip, evitando la duplicazione della stessa informazione in due punti diversi della configurazione. Con tre nodi aggregator attivi, la verifica sperimentale (si veda la sezione dei risultati) mostra una distribuzione dei client sostanzialmente uniforme fra i tre nodi.

C. Sincronizzazione dello stato tramite gossip

Poiché lo sharding fissa un client a un solo aggregator, il gossip non serve a coordinare le decisioni di rate limiting in tempo reale, bensì a garantire che lo stato dei bucket sia disponibile anche sugli altri nodi in caso di guasto del nodo titolare. Ogni aggregator, a intervalli regolari (5 secondi, valore configurabile), invia agli altri nodi aggregator lo stato dei bucket modificati più di recente. Alla ricezione, ciascun nodo confronta il timestamp di ultimo aggiornamento del bucket ricevuto con quello eventualmente già presente in locale e mantiene la versione più recente (politica last-writes). In questo modo, se il nodo titolare di un client diventa irraggiungibile, un nodo di backup dispone comunque di uno stato recente — anche se non necessariamente aggiornato all'ultima richiesta — su cui continuare a operare.

D. Circuit breaker lato edge

Per gestire i guasti dei nodi aggregator, l'edge implementa un circuit breaker per ciascuna destinazione, con i tre stati classici Closed, Open e Half-Open. In stato Closed le richieste vengono inoltrate normalmente; ogni fallimento (timeout o errore di connessione gRPC) viene contato, e al superamento di una soglia di fallimenti consecutivi (3, nella configurazione corrente) il circuito passa in stato Open, interrompendo temporaneamente l'invio di richieste verso quel nodo e reindirizzandole verso il nodo successivo disponibile nell'anello di hashing. Trascorso un periodo di raffreddamento (5 secondi), il circuito passa in stato Half-Open e lascia passare una richiesta di prova: se ha successo il circuito torna Closed, altrimenti torna Open e il raffreddamento riparte.

E. Messaggistica asincrona verso analytics

Ogni volta che una richiesta viene rifiutata, l'edge pubblica un evento di violazione su un broker NATS, in modo disaccoppiato rispetto al flusso sincrono di risposta al client:

la pubblicazione non blocca né rallenta la risposta HTTP. Il nodo analytics si sottoscrive allo stesso soggetto NATS, riceve gli eventi in modo asincrono e mantiene, per ciascun client, un conteggio dei rifiuti in una finestra temporale fissa di 60 secondi: un unico timer condiviso azzerà i contatori di tutti i client contemporaneamente allo scadere della finestra; al superamento di una soglia (5 rifiuti nella finestra) genera un alert, una sola volta per client per finestra. Questo disaccoppiamento consente al sistema di raccogliere dati diagnostici e di attivare notifiche senza introdurre alcuna dipendenza sincrona tra il percorso critico delle richieste e il livello di analisi.

F. Tracciamento distribuito

Trasversalmente ai pattern precedenti, il sistema integra il tracciamento distribuito tramite OpenTelemetry, con esportazione delle tracce verso Jaeger. Ogni nodo inizializza un proprio tracer provider all'avvio; le chiamate gRPC sincrone tra edge e aggregator propagano automaticamente il contesto di traccia tramite gli stats handler di OpenTelemetry per gRPC, mentre nel percorso asincrono l'edge inietta l'header di propagazione W3C (traceparent) come campo aggiuntivo del payload JSON pubblicato su NATS; l'analytics lo estrae alla ricezione per ricollegare lo span di elaborazione della violazione alla traccia originaria della richiesta HTTP. Questo permette di seguire una singola richiesta end-to-end anche quando attraversa un canale sincrono e uno asincrono, cosa che sarebbe altrimenti impossibile osservare guardando i soli log dei singoli nodi.

IV. PIATTAFORMA SOFTWARE, LIBRERIE E TESTING

A. Linguaggio e librerie principali

L'intero sistema è scritto in Go, scelto per il supporto nativo alla concorrenza (goroutine e channel), utile sia nell'implementazione dei nodi sia nella scrittura degli scenari di carico, e per l'ecosistema maturo di librerie per gRPC e Protobuf. Le dipendenze principali del progetto sono: google.golang.org/grpc e google.golang.org/protobuf per la comunicazione sincrona tra edge e aggregator; github.com/nats-io/nats.go per la messaggistica asincrona verso il nodo analytics; le librerie go.opentelemetry.io/otel (SDK, [otelgrpc](https://go.opentelemetry.io/otel/sdk/otelgrpc), [otelhttp](https://go.opentelemetry.io/otel/sdk/otelhttp), esportatore OTLP su gRPC) per il tracciamento distribuito. Il progetto è organizzato come un unico modulo Go con un package per ciascun tipo di nodo (edge, aggregator, analytics) più alcuni package condivisi (internal/config, internal/events, internal/tracing) che raccolgono rispettivamente la configurazione centralizzata, le strutture dati degli eventi scambiati via NATS e l'inizializzazione del tracciamento, evitando duplicazioni tra i nodi.

B. Comunicazione tra i nodi

L'interfaccia sincrona tra edge e aggregator è definita tramite Protobuf ([proto/ratelimiter.proto](https://github.com/nats-io/nats.go/blob/main/otel/otelhttp/otelhttp.pb.go)) e generata come servizio gRPC, con un unico metodo (CheckQuota) che riceve l'identificativo del client e restituisce l'esito (consentito/rifiutato) più i token residui. La scelta di gRPC risponde al requisito di comunicazione sincrona basata su RPC richiesto dal progetto, ed è stata preferita a un'interfaccia REST per l'overhead ridotto e per il supporto nativo alla propagazione del contesto di traccia tramite gli stats handler di OpenTelemetry. Il canale asincrono verso analytics, invece, utilizza NATS come message broker: l'edge pubblica un evento JSON per ogni richiesta rifiutata su un soggetto dedicato, e analytics vi si sottoscrive in modo indipendente,

senza che i due nodi debbano conoscersi direttamente o essere entrambi disponibili nello stesso istante.

C. Containerizzazione e orchestrazione

Ogni nodo è pacchettizzato in un'immagine Docker tramite build multi-stage, che compila il binario Go in uno stage separato e produce un'immagine finale minimale. Il deployment principale del sistema è realizzato con Docker Compose, che orchestra i tre aggregator, l'edge, l'analytics, il broker NATS e Jaeger in un'unica rete, con la configurazione (indirizzi degli aggregator, URL di NATS, endpoint OTLP) iniettata tramite variabili d'ambiente anziché cablata nelle immagini. Come percorso alternativo, il progetto include anche un insieme di manifest Kubernetes (namespace, ConfigMap, Deployment e Service per ciascun nodo) pensati per un cluster locale; questa seconda modalità non sostituisce Docker Compose ma dimostra la portabilità della stessa configurazione applicativa su un'infrastruttura di orchestrazione differente, con la configurazione condivisa tramite ConfigMap invece che tramite variabili d'ambiente statiche.

D. Deployment su cloud

Per la verifica su un'infrastruttura reale, il sistema è stato distribuito su un'istanza AWS EC2 (Ubuntu), usando Docker Compose come sull'ambiente locale. L'accesso ai servizi esposti pubblicamente (edge e analytics) è regolato tramite i Security Group dell'istanza, mentre i servizi non destinati a un uso esterno — in particolare l'interfaccia di Jaeger — restano raggiungibili solo dalla rete interna della macchina, e sono ispezionabili da remoto tramite tunnel SSH anziché tramite esposizione diretta della porta. Questa scelta riflette il principio per cui solo l'interfaccia dei nodi realmente destinata ai client (edge) e all'interrogazione dei dati aggregati (analytics) deve essere pubblicamente raggiungibile.

E. Strategia di testing

Il testing del progetto è organizzato come segue: è presente un package loadtest dedicato, con sei scenari indipendenti eseguiti come programmi Go a sé stanti: distribuzione dei client tra gli aggregator (sharding), comportamento sotto concorrenza crescente, latenza al crescere del numero di client, propagazione dello stato tramite gossip, sensibilità della soglia di alert a profili di traffico onesti e aggressivi, e comportamento del sistema durante il guasto simulato di un nodo aggregator (failover). Ogni scenario produce un file CSV con i dati grezzi o aggregati della prova, usati poi per l'analisi quantitativa presentata nella sezione successiva.

V. RISULTATI DEI TEST

Come richiesto dalla consegna, questo scenario valuta esplicitamente il compromesso tra la precisione del rate limiting (consistenza dei contatori distribuiti) e l'overhead di latenza iniettato sul traffico client legittimo, al crescere del numero di richieste. Nel sistema realizzato questo compromesso non è visibile in condizioni normali — lo sharding garantisce che un client sia sempre gestito dallo stesso aggregator, quindi il suo contatore non è mai conteso tra nodi diversi — ma emerge nelle due situazioni in cui la proprietà del contatore può cambiare nodo: la replica asincrona tramite gossip e il failover in caso di guasto. I tre esperimenti seguenti misurano separatamente le due dimensioni del compromesso e la loro combinazione in condizioni di guasto reale.

A. Overhead di latenza al crescere del numero di richieste

N client	p50 (ms)	p95 (ms)	p99 (ms)	errori
10	14.61	33.50	33.50	0
50	64.41	89.78	92.74	0
200	314.22	434.28	449.39	0
1000	1473.50	1967.71	2029.04	0

Aumentando il numero di client concorrenti di due ordini di grandezza (da 10 a 1000), la latenza mediana cresce di un fattore paragonabile ($\sim 100\times$), senza mostrare un'esplosione super-lineare: il partizionamento su tre nodi aggregator limita la contesa che ciascun nodo deve assorbire. Lo scarto assoluto tra p50 e p99 si allarga marcatamente al crescere del carico (da ~ 19 ms a $N=10$ a ~ 556 ms a $N=1000$), segno di code di attesa più lunghe sulle connessioni gRPC condivise. Il dato più rilevante ai fini del compromesso richiesto è però il conteggio degli errori, sempre pari a zero: in assenza di guasti, il costo della scalabilità si manifesta esclusivamente come overhead di latenza, senza alcuna perdita di precisione nelle decisioni di allow/reject.

B. Precisione dei contatori distribuiti: ritardo di propagazione via gossip

Per isolare la componente di precisione, ogni prova usa un client mai visto prima: il suo bucket viene prima svuotato completamente sul nodo primario (5 richieste consecutive), poi, dopo un'attesa wait_ms, viene interrogato una sola volta il nodo secondario. Se il gossip ha già propagato lo stato svuotato, il secondario risponde con un numero di token residui inferiore a quello di un bucket mai toccato (capacity - 1 = 4); altrimenti il secondario, non avendo ricevuto nulla, tratta il client come nuovo e restituisce il valore di un bucket fresco.

Quando il gossip ha già propagato lo stato (10 client indipendenti)



Il punto in cui lo stato risulta replicato non cresce in modo monotono con l'attesa: a 2000 ms risulta già propagato, a 3000 ms no. Questo è coerente con il funzionamento del gossip, che gira su un ticker periodico (5 s) indipendente per ciascun nodo aggregator e non sincronizzato né con l'orologio del test né tra i nodi stessi: se la prova cade poco prima del prossimo invio, il secondario non ha ancora ricevuto nulla, se cade poco dopo l'ha già ricevuto. Una volta che la replica è avvenuta, invece, i token residui osservati crescono in modo coerente con il tempo trascorso ($0 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 2$), poiché il ricarico dei token continua a essere calcolato a partire dal timestamp propagato dal nodo primario. Questo esperimento quantifica direttamente il lato "precisione" del compromesso: per una finestra fino a circa 5 secondi da un cambiamento di stato, un nodo che non possiede il client può rispondere con un contatore non ancora aggiornato.

C. Il compromesso in condizioni di guasto: failover

Lo scenario invia una richiesta al secondo per un singolo client per 60 secondi; al secondo 15 il nodo primario di quel client viene arrestato (docker compose stop), al secondo 40

viene riavviato. L'estratto seguente copre il momento del guasto:

t (s)	esito	allowed	degraded	latenza (ms)
16	200	True	False	5.64
17	429	False	False	2.39
18	200	True	False	1009.94
19	200	True	False	1005.38
20	200	True	False	1007.43
21	429	False	False	7.17
22	200	True	False	4.72

Fino al secondo 17 le richieste alternano tra consentite e rifiutate con latenze dell'ordine di pochi millisecondi: un comportamento atteso, dato che il tasso di richieste (1/s) è il doppio del tasso di ricarica dei token (0.5/s). Subito dopo l'arresto del nodo primario, tre richieste consecutive ($t=18-20$) impiegano circa un secondo ciascuna — il tempo dei tentativi verso il nodo ormai irraggiungibile prima che il circuit breaker registri i fallimenti e reindirizzi la richiesta al nodo successivo nell'anello — ma in tutti e tre i casi la risposta finale resta un 200 con degraded=false: il client ottiene comunque una decisione corretta, pagata però con un forte overhead di latenza. Dal secondo 21 in poi la latenza torna ai valori normali, segno che il circuito è ormai aperto verso il nodo guasto e le richieste successive vengono inoltrate direttamente al nodo di backup senza ritentare quello caduto; il riavvio al secondo 40 non produce alcun picco di latenza visibile (per questo non è stato mostrato), poiché il traffico era già gestito correttamente dal nodo di backup. In nessuna delle 60 richieste il sistema è passato in stato degraded: con tre nodi aggregator, il fallimento di uno solo lascia sempre disponibile un nodo capace di rispondere in modo consistente.

D. Riflessione sui primi tre test

Nel complesso, i tre esperimenti mostrano il compromesso richiesto dalla consegna in due regimi distinti: in assenza di guasti, la scalabilità ha un costo puramente di latenza (V.A), a precisione invariata; in presenza di un guasto, il sistema sceglie esplicitamente la disponibilità (una risposta viene sempre fornita, mai in modalità degradata) a fronte di un breve intervallo di latenza elevata (V.C) e di una finestra di alcuni secondi in cui, se il client fosse dirottato su un nodo di backup, il suo contatore potrebbe non essere ancora sincronizzato (V.B).

E. Test aggiuntivi

Distribuzione dei client tra gli aggregator (sharding).

Su un campione di 2000 client, la ripartizione tra i tre nodi è 34.00% / 33.10% / 32.90%, molto vicina alla distribuzione uniforme attesa e a conferma della correttezza dell'hashing FNV-1a usato per l'instradamento.

Comportamento sotto concorrenza. Inviando K richieste simultanee per lo stesso client contro un bucket di capacità 5:

K	Consentite	Rifiutate	Eccesso
5	5	0	0
20	5	15	0

50	5	45	0
200	5	195	0

Il numero di richieste consentite resta sempre esattamente pari alla capacità del bucket, con zero superamenti anche a 200 richieste simultanee: il lock sul bucket garantisce la correttezza del conteggio anche sotto forte concorrenza sullo stesso shard.

Sensibilità della soglia di alert. Con soglia di allarme impostata a 5 rifiuti in una finestra di 60 secondi, un profilo di traffico "onesto" (una richiesta ogni 2200 ms, compatibile col tasso di ricarica) produce 0 rifiuti nella finestra e non genera alcun alert, mentre un profilo aggressivo (una richiesta ogni 50 ms) ne produce 1039, ben oltre la soglia: il meccanismo di allerta distingue correttamente un uso legittimo da un abuso sostenuto, senza falsi positivi sul traffico onesto.

VI. LIMITAZIONI RISCONTRATE

Finestra di inconsistenza durante il failover. La replica tramite gossip è per costruzione eventually consistent: come misurato nella sezione precedente, un nodo di backup può impiegare fino a circa 5 secondi (l'intervallo del gossip) per ricevere lo stato aggiornato di un bucket. Se un client viene dirottato su un nodo di backup proprio in questa finestra, la decisione di allow/reject viene presa su uno stato potenzialmente non aggiornato, con il rischio — non osservato negli esperimenti condotti, ma non escluso in linea di principio — di concedere temporaneamente più richieste della capacità nominale a un client il cui nodo primario è appena caduto.

Overhead di latenza al momento del guasto. Il rilevamento di un nodo non più raggiungibile da parte del circuit breaker non è istantaneo: le prime richieste successive al guasto pagano il costo pieno del timeout gRPC (circa un secondo nei test condotti) prima che il circuito si apra e la richiesta venga reindirizzata. Il sistema privilegia la possibilità di completare la richiesta tramite il nodo corrente rispetto alla rapidità di rilevamento del guasto, accettando un aumento della latenza nella fase transitoria, ma il costo — per quanto limitato a poche richieste — resta un compromesso esplicito e non un comportamento privo di conseguenze per il client.

Stato mantenuto solo in memoria. Sia i bucket degli aggregator sia le tracce raccolte da Jaeger vivono esclusivamente in memoria, senza alcuna persistenza su disco o storage esterno. Il riavvio di un nodo aggregator azzerà lo stato dei bucket di cui era proprietario: se non è stato oggetto di un gossip recente verso gli altri nodi, quello stato non è recuperabile e il client riparte con una quota piena. Analogamente, un riavvio di Jaeger comporta la perdita di tutte le tracce raccolte fino a quel momento.

Consegna at-most-once degli eventi di violazione. La pubblicazione degli eventi di violazione su NATS non utilizza meccanismi di persistenza o conferma (JetStream); se il nodo analytics è temporaneamente non raggiungibile nel momento in cui un evento viene pubblicato, quell'evento va perso senza possibilità di essere ritrasmesso. Il livello di analytics riflette quindi un conteggio best-effort dei rifiuti, non una garanzia di completezza.

Composizione statica del cluster. L'elenco dei nodi aggregator è definito a configurazione (AGGREGATOR_ADDRESSES) e condiviso da tutti i nodi all'avvio: aggiungere o rimuovere un aggregator richiede di aggiornare la configurazione e riavviare edge e aggregator coinvolti. Non è presente alcun meccanismo di service discovery o di membership dinamica, per cui la scalabilità orizzontale del livello aggregator non è realizzabile a runtime.

Copertura sperimentale limitata a un singolo guasto e a un singolo edge. Gli scenari di test condotti considerano il guasto di un solo nodo aggregator alla volta e una singola istanza del nodo edge; non sono stati verificati sperimentalmente né il comportamento in presenza di guasti multipli simultanei, né la coerenza dello stato del circuit breaker tra più istanze edge eseguite in parallelo, scenario comunque compatibile con l'architettura ma non incluso nella campagna di test svolta.

Assenza di autenticazione e autorizzazione. Né l'interfaccia HTTP dell'edge e dell'analytics né il servizio gRPC tra edge e aggregator richiedono alcuna forma di autenticazione: l'identificazione del client si basa sul solo identificativo applicativo passato nella richiesta, senza verifica della sua autenticità. Questo aspetto è stato considerato fuori dall'ambito del progetto, incentrato sui pattern architetturali e sulla logica di rate limiting distribuito.

REFERENZE

- [1] A. Demers, D. Greene, C. Hauser, W. Irish, J. Larson, S. Shenker, H. Sturgis, D. Swinehart, D. Terry, "Epidemic Algorithms for Replicated Database Maintenance," Proceedings of the 6th ACM Symposium on Principles of Distributed Computing (PODC), 1987.
- [2] M. T. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed., Pragmatic Bookshelf, 2018
- [3] Synadia, NATS Documentation. [Online]. Available: <https://docs.nats.io/>
- [4] Cloud Native Computing Foundation, OpenTelemetry Documentation. [Online]. Available: <https://opentelemetry.io/docs/>
- [5] [Cloud Native Computing Foundation, Jaeger: Open Source, End-to-End Distributed Tracing Documentation. [Online]. Available: <https://www.jaegertracing.io/docs/>